



Faculté des sciences et de génie
Département de génie électrique et de
génie informatique,
Université Laval



Grenoble INP - PHELMA

BOUCHUT Tanguy

Filière Systèmes Embarqués et Objets Connectés (SEOC)
Année universitaire 2025-2026

Exécution persistante de tâches multi-robots avec garanties de sécurité via Control Barrier Functions et évitement d'obstacles

du 18/05/2026 au 04/09/2026

Lieu du stage :

Laboratoire de vision et systèmes numériques (LVSN)
Pavillon Adrien-Pouliot
1065, avenue de la Médecine
Québec (Québec) G1V 0A6, CANADA

Sous la supervision de :

Maître de stage : Hanqing Zhao, hanqing.zhao@gel.ulaval.ca

Tuteur école : Benjamin PEYRILLE, benjamin.peyrille@grenoble-inp.fr

Confidentialité : ☐ oui ☒ non

Remerciements

Je tiens à exprimer ma profonde gratitude à **Hanqing** pour m'avoir proposé ce stage et m'avoir accordé sa confiance pour sa réalisation. Son accueil chaleureux et son encadrement tout au long de ce stage ont été précieux pour ma réussite.

Je remercie également **Hassan**, pour avoir suivi de près l'avancement de mes travaux et pour ses conseils avisés grâce à son expertise dans le domaine.

Pour finir, je souhaite également remercier l'ensemble de l'équipe du Laboratoire de vision et systèmes numériques (LVSN) pour leur bienveillance et l'atmosphère chaleureuse régnant au sein du laboratoire.

Un grand merci à tous.

Table des matières

Remerciements	i
Glossaire et Acronymes	iii
Liste des figures	iii
Liste des tableaux	iii
Introduction	1
1 Conception d'un simulateur multi-robots dans un environnement partiellement observable	2
1.1 Modélisation des robots et interactions inter-robots	2
1.2 Modélisation de l'environnement et perception sous observabilité partielle .	5
1.3 Gestion séquentielle et synchrone des pas de temps	8
1.4 Architecture logicielle modulaire et assurance qualité	11
2 Planification de missions persistantes dans un environnement partiellement observable	14
2.1 Architecture décentralisée et cycle de vie de la mission persistante	14
2.2 Planification spatio-temporelle sur graphe local et repli global	16
2.3 Conception des fonctions barrières de contrôle (CBF)	19
2.4 Campagnes expérimentales, analyse critique et feuille de route	22
3 Conclusion	24
Références	26
A Auto-évaluation des compétences (Référentiel Phelma)	27
B Considération des impacts écologiques et sociétaux	32
B.1 Analyse de l'impact environnemental et sociétal	32

Glossaire et Acronymes

CBF : *Control Barrier Functions*.

OGM : *Occupancy Grid Map* (Grille).

SDF : *Signed Distance Field* (Distance).

RRG : *Rapidly-exploring Random Graph*.

QP : *Quadratic Program* (Projection).

FRA* : *Fringe-Retrieving A** (Repli).

MR / PR : *Mission Robot / Pursuer Robot*.

Buzz / Zenoh : Coordination et réseau.

IPC : Tubes de synchronisation temporelle.

HAL : *Hardware Abstraction Layer*.

Table des figures

1	Représentation spatiale de l'environnement et champ de distance SDF . . .	7
2	Chronogramme de synchronisation synchrone (<i>lockstep</i>) par barrière IPC .	10
3	Architecture logicielle modulaire en couches et flux d'orchestration	12
4	Machine à états finis distribuée pour la persistance de mission	15
5	Échantillonnage probabiliste biaisé pour la génération du graphe RRG . . .	17
6	Visualisation et diagnostic de sélection de trajectoire sur graphe	19

Liste des tableaux

1	Paramètres nominaux de la plateforme de simulation	5
2	Synthèse du balayage paramétrique des poids de trajectoire	23
3	Synthèse de l'empreinte carbone estimée sur la période de stage (16 semaines). .	32

Introduction

Le déploiement de systèmes robotiques autonomes dans des environnements hostiles ou difficilement accessibles représente un enjeu stratégique majeur pour la surveillance environnementale, l'inspection d'infrastructures critiques et l'exploration de territoires isolés. Cette problématique est particulièrement prégnante au Québec, dont l'immensité du territoire et la rigueur climatique imposent le recours à des solutions automatisées et pérennes.

Dans ce contexte, le Laboratoire de Vision et Systèmes Numériques (LVSN) de l'Université Laval mène des recherches avancées sur la perception et la coordination multi-robots. L'un des défis scientifiques majeurs réside dans la réalisation de missions « persistantes », où une observation continue doit être assurée sur de longues durées malgré l'autonomie énergétique limitée des vecteurs [1, 2]. Une telle exigence impose une rotation coordonnée des robots (relève sur base de recharge) et un évitement rigoureux des collisions dans un espace encombré sous observabilité partielle (portée limitée des capteurs et méconnaissance *a priori* des obstacles).

Dès lors, la question centrale de cette étude se formule ainsi : *comment garantir formellement la sécurité énergétique et la continuité opérationnelle d'une flotte multi-robots se relayant de façon décentralisée et autonome au sein d'un environnement partiellement observable ?*

Pour y répondre, notre démarche articule une architecture décentralisée de coordination d'essaim, une planification spatio-temporelle de trajectoires sur graphe local et des garanties formelles de sécurité fondées sur le cadre théorique des *Control Barrier Functions* (CBF) [3]. Réalisé sur seize semaines au LVSN, ce stage de 2^e année Phelma s'articule autour de deux volets :

1. Concevoir et développer une plateforme de simulation multi-robots synchrone et modulaire, intégrant cinématique, dynamique de batterie, cartographie locale (OGM/SDF) et ordonnancement déterministe strict ;
2. Développer un cadre de planification et de contrôle pour missions persistantes, combinant coordination décentralisée par partage d'état, recherche de trajectoires sur graphe local et sécurité par CBF, étayé par une analyse critique des limites du prototype intermédiaire.

Le présent rapport rend compte de ces développements et s'articule en deux parties principales :

- La section 1 détaille la **conception, la modélisation et l'architecture du simulateur multi-robots**, dont l'implémentation complète et la démarche d'assurance qualité logicielle constituent le socle technique abouti du stage ;
- La section 2 formalise la **planification de missions persistantes avec garanties de sécurité** (FSM décentralisée, scoring multi-objectifs, filtrage de sécurité CBF) et présente une évaluation expérimentale lucide identifiant les verrous techniques du couplage en boucle fermée ;
- Enfin, la section 3 dresse le **bilan** de ces réalisations et dégage les perspectives vers des flottes physiques.

1 Conception d'un simulateur multi-robots dans un environnement partiellement observable

L'évaluation et la validation d'algorithmes de coordination multi-robots avec garanties formelles de sécurité énergétique constituent une étape critique qui ne peut être menée directement sur des plateformes matérielles réelles sans risque majeur d'avarie. En effet, tester en conditions réelles des scénarios limites de collision, de perte de visibilité ou d'épuisement critique de batterie sur une flotte de drones aériens s'avère à la fois prohibitif en termes de coût et dangereux sur le plan expérimental.

Dès lors, le développement d'une plateforme de simulation instrumentée s'impose comme un prérequis indispensable. Bien que le simulateur multi-robots de référence au laboratoire soit *ARGoS* [4], sur lequel l'intégration finale des algorithmes a vocation à être déployée, le choix s'est porté sur le développement préalable d'un simulateur sur-mesure en Python. Ce choix a été motivé par la rapidité de prototypage, la facilité d'implémentation et la richesse de l'écosystème scientifique (manipulation matricielle vectorisée des grilles OGM, calcul continu des champs SDF avec SciPy, et exécution automatisée de bancs de tests intensifs en mode *headless*). Ce simulateur en Python est directement couplé à la machine virtuelle distribuée du langage d'essaim *Buzz* [5] (langage spécialement conçu pour les essaims de robots) via une passerelle d'interopérabilité Cython (*PyBuzz*), préservant ainsi la compatibilité totale du code de coordination avec *ARGoS*.

Cette première section détaille la conception et la réalisation de cette infrastructure de simulation. Sont successivement présentées la modélisation cinématique, énergétique et sensorielle des robots, la génération procédurale des environnements et de l'observabilité partielle, le moteur d'ordonnancement séquentiel garantissant un synchronisme strict en temps discret, et enfin l'architecture logicielle globale du simulateur ainsi que sa démarche d'assurance qualité.

1.1 Modélisation des robots et interactions inter-robots

La fidélité de la simulation repose en premier lieu sur une formalisation mathématique rigoureuse des robots et de leurs contraintes opérationnelles. Dans la suite de ce document, le terme **robot** (ou drone) désigne l'unité mobile autonome dans son intégralité, intégrant la plateforme physique simulée (cinématique, encombrement, batterie) et ses processus logiciels de décision et de contrôle. Dans ce projet, la flotte est composée de drones aériens quadrirotors évoluant dans un plan horizontal bidimensionnel à altitude de vol constante.

1.1.1 Modèle cinématique et contraintes d'actionnement

Dans le cadre de cette étude, les drones évoluent dans un plan horizontal à altitude constante et à vitesse modérée. Chaque robot $i \in \{1, \dots, N\}$ est ainsi modélisé sous l'hypothèse standard d'un système cinématique holonome du premier ordre (intégrateur simple), recevant directement une consigne de vitesse dans le plan.

L'état spatial du robot est défini par son vecteur position dans le repère global $\mathbf{p}_i(t) = [x_i(t), y_i(t)]^\top \in \mathbb{R}^2$. L'actionneur virtuel applique directement une consigne de vitesse $\mathbf{u}_i(t) = [v_{x,i}(t), v_{y,i}(t)]^\top \in \mathbb{R}^2$. L'évolution temporelle continue du robot est régie par l'équation différentielle suivante, bornée par la vitesse maximale admissible du drone $v_{\max}$:

$$\dot{\mathbf{p}}_i(t) = \mathbf{u}_i(t), \quad \text{avec} \quad \|\mathbf{u}_i(t)\| \leq v_{\max} \quad (1)$$

Dans le cadre de la simulation numérique, ce modèle est discrétisé au pas d'échantillonnage temporel Δt (associé à une fréquence d'horloge de simulation $f = \frac{1}{\Delta t}$) selon un schéma d'intégration d'Euler explicite :

$$\mathbf{p}_i(t + \Delta t) = \mathbf{p}_i(t) + \mathbf{u}_i(t) \cdot \Delta t \quad (2)$$

Chaque robot possède un encombrement physique modélisé par un disque protecteur de rayon $r_{\text{drone}} = 0,256$ m (gabarit physique calibré sur les spécifications du quadrirotor *ModalAI Seeker* utilisé dans la plateforme expérimentale du laboratoire [1,6]). La condition stricte de non-collision physique entre deux robots distincts i et j impose une distance inter-centres supérieure au diamètre critique de collision :

$$\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \geq d_{\text{coll}} = 2 \cdot r_{\text{drone}} = 0,512 \text{ m}, \quad \forall i \neq j \quad (3)$$

Puisque le modèle cinématique d'intégrateur simple fait abstraction de l'inertie et des limites d'accélération transitoires du drone réel, une marge de sécurité supplémentaire $d_{\text{safe}} > d_{\text{coll}}$ est introduite. Cette marge garantit que les décélérations d'urgence et les discontinuités de trajectoire n'entraînent aucune violation de l'enveloppe physique d_{coll} .

1.1.2 Dynamique énergétique et modèle linéaire de batterie

Dans le simulateur, la décharge de la batterie en vol est modélisée par une consommation de puissance constante, indépendante de la vitesse de translation. En effet, pour un quadrirotor évoluant à des vitesses modérées, l'essentiel de l'énergie consommée sert à maintenir l'appareil en l'air face à la gravité (sustentation), l'effort supplémentaire requis pour la translation horizontale restant négligeable en comparaison.

Cette hypothèse d'une décharge linéaire dans le temps est largement adoptée et validée dans la littérature robotique :

- Dorling *et al.* [7] ont dérivé et validé expérimentalement ce modèle linéaire sur multirotors, démontrant que la puissance électrique appelée reste sensiblement identique en vol stationnaire (*hover*) et en translation à vitesse constante ;
- Fouad, Zhao et Beltrame [1] confirment expérimentalement cette invariance sur des quadrirotors réels (*ModalAI Seeker*) en mission de surveillance persistante, où l'énergie résiduelle mesurée présente une décroissance affine strictement linéaire en temps dans des conditions d'air calme et avec des mouvements non agressifs ;
- Ce comportement linéaire est également au cœur des formulations d'autosuffisance et d'autonomie énergétique pour drones quadrirotors établies par Fouad et Beltrame [2,

8].

Chaque drone dispose d'une batterie de capacité nominale maximale $E_{\max}$. L'état d'énergie résiduelle $E_i(t) \in [0, E_{\max}]$ évolue selon une loi strictement linéaire en temps :

$$\dot{E}_i(t) = \begin{cases} -P_{\text{discharge}}, & \text{en phase de vol (sustentation et translation)} \\ +P_{\text{charge}} = +\gamma \cdot P_{\text{discharge}}, & \text{posé sur la station de recharge} \end{cases} \quad (4)$$

où $\gamma > 1$ représente le facteur d'accélération de la recharge par rapport au taux de décharge en vol.

Cette linéarité temporelle présente un intérêt mathématique déterminant pour la planification : pour une trajectoire planifiée de longueur géométrique L parcourue à la vitesse de consigne de croisière v_{cruise} , la durée de parcours s'exprime par $t_{\text{vol}} = \frac{L}{v_{\text{cruise}}}$. Par conséquent, l'énergie minimale requise pour franchir cette distance s'exprime directement par le produit de la puissance de décharge et du temps de vol :

$$E_{\text{transit}}(L) = P_{\text{discharge}} \cdot \frac{L}{v_{\text{cruise}}} \quad (5)$$

Cette relation analytique directe constitue le pivot du calcul des réserves de retour et des fonctions barrières énergétiques présentées dans la section 2.

Échelle de simulation et choix des paramètres énergétiques. Les paramètres nominaux retenus ($E_{\max} = 1000$ J et $P_{\text{discharge}} = 23$ W, correspondant à une autonomie nominale en vol continue de l'ordre de 43,5 s) constituent une **mise à l'échelle temporelle délibérée**. Ce choix méthodologique permet d'observer et d'évaluer de multiples cycles complets de relève opérationnelle (décollage, poursuite, surveillance active, retour base et recharge complète) au cours d'expérimentations de quelques minutes de calcul simulé, sans altérer la représentativité des lois de commande adimensionnées.

1.1.3 Paramètres nominaux de simulation et configurabilité

Afin de garantir la modularité logicielle et de découpler les algorithmes des grandeurs physiques, aucun paramètre n'est codé en dur dans le code source : l'ensemble des constantes des robots et de l'environnement est externalisé dans un fichier de configuration unique (`config.json`), respectant le principe de séparation entre la configuration et le code. Au lancement, ce fichier est validé par la structure Python `SimulationConfig` puis répercuté automatiquement sur les constantes de la machine virtuelle Buzz (`config.bzz`).

Le tableau 1 consigne le jeu de paramètres nominaux retenu pour caractériser la flotte au sein de la plateforme de simulation.

Ayant formalisé les caractéristiques cinématiques et énergétiques des robots, la sous-section suivante aborde la modélisation de l'environnement opérationnel et les mécanismes de cartographie locale sous observabilité partielle.

Paramètre physique	Symbole	Valeur	Unité / Commentaire
Taille de l'essaim	N	3	Drones simultanés en simulation
Fréquence de simulation	f	10,0	Hz (pas temporel $\Delta t = 0,1$ s)
Vitesse maximale admissible	$v_{\max}$	5,0	m/s (saturation de vitesse)
Vitesse de croisière nominale	v_{cruise}	2,0	m/s (vitesse nominale de transit)
Rayon physique du drone	r_{drone}	0,256	m (gabarit <i>ModalAI Seeker</i> [6])
Distance critique de collision	d_{coll}	0,512	m ($2 \cdot r_{\text{drone}}$)
Marge de sécurité protectrice	d_{safe}	0,700	m (anticipation cinématique)
Capacité nominale de batterie	$E_{\max}$	1000	J (énergie maximale d'échelle)
Puissance de décharge en vol	$P_{\text{discharge}}$	23,0	W (J/s, sustentation et vol)
Facteur de recharge à la base	γ	2,0	Puissance de charge 46,0 W
Portée maximale du LiDAR	R_{lidar}	10,0	m (360 faisceaux à 360°)
Résolution spatiale de la grille	δ	0,10	m/cellule

TABLE 1 – Jeu de paramètres nominaux de la plateforme de simulation multi-robots (`config.json`).

1.2 Modélisation de l'environnement et perception sous observabilité partielle

Pour éprouver la robustesse des algorithmes de navigation et de sécurité dans des conditions représentatives, l'environnement simulé articule une double représentation spatiale : une vérité terrain continue et vectorielle pour la physique du monde, et une perception locale discrète et bruitée sous observabilité partielle pour chaque robot.

1.2.1 Environnement global et vérité terrain procédurale

À ce stade des travaux, la simulation est restreinte à un environnement bidimensionnel (2D) afin de simplifier le problème : ce choix délibéré permet de se concentrer sur la conception de l'architecture et de valider le principe fondamental des algorithmes de contrôle et d'évitement de collisions avant d'envisager une extension tridimensionnelle. L'arène est ainsi délimitée par un périmètre rectangulaire fermé et peuplée d'obstacles statiques géométriques (parois polygonales, piliers rectangulaires orientés et colonnes circulaires), modélisés sous forme vectorielle continue à l'aide de la bibliothèque géométrique *Shapely*.

Un outil dédié (`generate_random_map.py`) génère des configurations de cartes selon une distribution paramétrable en densité et en encombrement (par défaut, une couverture cible d'obstacles de 12 % sur une surface de 100×100 m²). Afin d'obtenir un taux d'occupation exact malgré les intersections d'obstacles aléatoires, le script calcule l'aire cumulée réelle par fusion booléenne (`shapely.ops.unary_union`). La configuration validée est sérialisée dans un fichier JSON (`current_map.json`), assurant la reproductibilité stricte des scénarios expérimentaux entre différentes variantes algorithmiques.

Pour garantir des performances de simulation compatibles avec un fonctionnement synchrone multi-robots, les requêtes géométriques globales sont accélérées par un index

spatial arborescent de type R-tree (`shapely.strtree.STRTree`). Cet index réduit la complexité de recherche des obstacles candidats lors du calcul des intersections de rayons de $O(M)$ à $O(\log M)$, où M est le nombre total d'obstacles.

1.2.2 Grille d'occupation probabiliste et perception LiDAR

Afin de respecter la contrainte d'observabilité partielle, les drones ne possèdent aucune connaissance *a priori* de la géométrie de l'arène. Chaque robot instancie sa propre grille d'occupation locale (*Occupancy Grid Map* – OGM) et la met à jour au fil de ses déplacements.

L'espace est discrétisé en cellules carrées de résolution fine $\delta = 0,1$ m. La carte locale est implémentée sous forme éparsée (table de hachage) : seules les cellules observées sont conservées en mémoire, les autres demeurant implicitement dans l'état *Inconnu*. Chaque cellule observée $\mathbf{c} = (j, k) \in \mathbb{Z}^2$ enregistre une vraisemblance en log-odds $L(\mathbf{c}) \in [l_{\min}, l_{\max}]$, mise à jour selon le modèle bayésien standard [9] :

$$\text{Modalité}(\mathbf{c}) = \begin{cases} \text{Inconnu (2)}, & \text{si } \mathbf{c} \notin \text{domaine exploré} \\ \text{Occupé (1)}, & \text{si } L(\mathbf{c}) > l_{\text{seuil}} = 0,0 \\ \text{Libre (0)}, & \text{si } L(\mathbf{c}) \leq l_{\text{seuil}} \end{cases} \quad (6)$$

Perception LiDAR et stratégies de mise à jour. Pour découvrir progressivement son environnement, chaque drone embarque un télémètre LiDAR 2D omnidirectionnel simulé émettant $N_{\text{rays}} = 360$ rayons uniformément sur 360° , avec une portée maximale $R_{\text{lidar}} = 10,0$ m. Les intersections entre les faisceaux laser et les obstacles continus sont calculées à chaque pas temporel via l'index spatial R-tree.

Pour projeter ces points d'impact continus sur la grille discrète, deux stratégies de mise à jour ont été développées (patron Stratégie) : un tracé de rayons classique fondé sur l'algorithme de Bresenham, et une méthode par tampon de profondeur angulaire (*DepthBufferStrategy*). Cette dernière comble les vides de discrétisation angulaire à longue portée et garantit une fermeture robuste des contours d'obstacles. Les masques de voisinage circulaire et les motifs de discrétisation sont précalculés et mis en cache mémoire à l'initialisation.

1.2.3 Champ de distance signée (SDF) et calcul des gradients

Si la grille d'occupation discrète est directement exploitable par les planificateurs de chemins sur graphe, la formulation des fonctions barrières de contrôle (CBF) requiert en revanche une fonction de distance $h(\mathbf{p})$ continue et différentiable dans tout l'espace d'état admissible.

Pour relier la perception discrète au contrôle continu, le module *SDFGenerator* extrait à chaque pas temporel une sous-grille locale centrée sur le drone et applique la **transformée de distance euclidienne** (*Euclidean Distance Transform* – EDT, via `scipy.ndimage.distance_transform_edt`), selon le formalisme *OGM-CBF* de Raja *et al.* [10].

Cette opération fournit un champ de distance euclidienne $D(\mathbf{p})$ associant à toute coordonnée continue $\mathbf{p} = [x, y]^\top$ la distance minimale la séparant du bord de l'obstacle le plus proche. Le gradient spatial $\nabla D(\mathbf{p})$ est approché numériquement par différences finies :

$$\nabla D(\mathbf{p}) = \begin{bmatrix} \frac{\partial D}{\partial x}(\mathbf{p}) \\ \frac{\partial D}{\partial y}(\mathbf{p}) \end{bmatrix} \quad (7)$$

Ce gradient spatial et le champ de distance constituent les entrées clés du filtre de sécurité cinématique CBF développé dans la section 2.

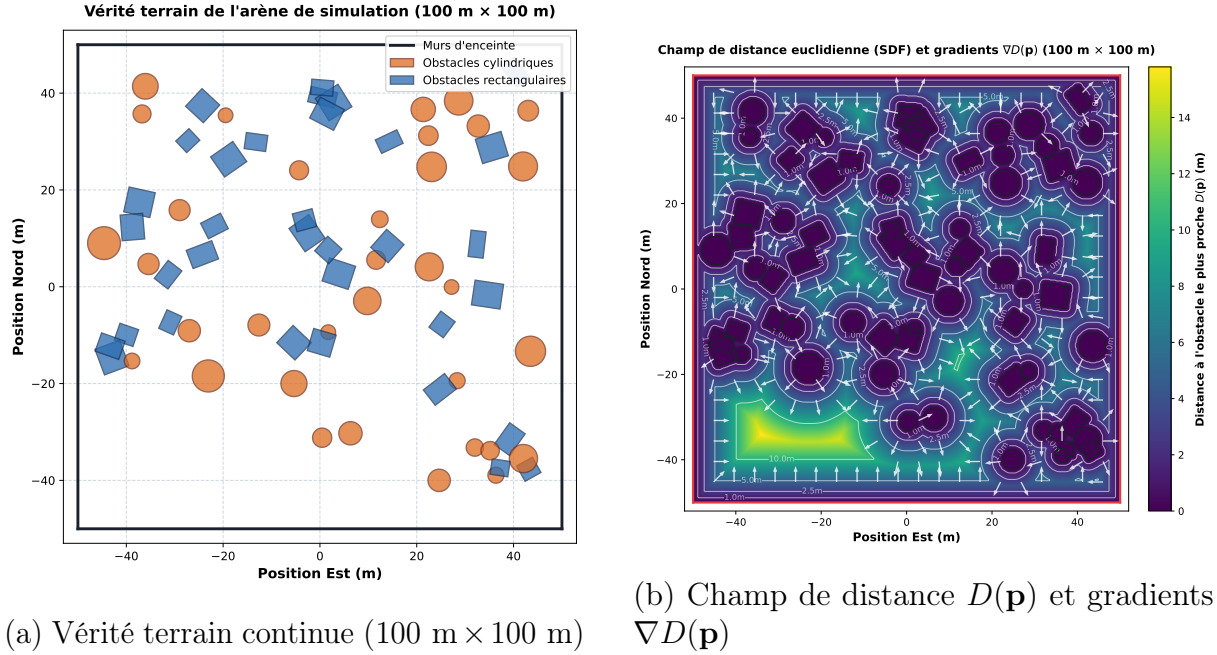


FIGURE 1 – Représentation spatiale de l'environnement de simulation. (a) Arène générée procéduralement avec obstacles géométriques continus (12 % de densité). (b) Champ de distance euclidienne $D(\mathbf{p})$ et gradient normalisé $\nabla D(\mathbf{p})$ calculés sur la grille d'occupation.

1.2.4 Validation métrologique de la cartographie et détection d'anomalies

Dans une simulation en boucle fermée, la validité des garanties de sécurité dépend strictement de la cohérence entre la vérité terrain géométrique et la carte locale construite par les capteurs. Deux types d'anomalies de discrétisation peuvent compromettre le système :

- **Les fuites géométriques (*Leaks*)** : cellules étiquetées à tort comme *Libres* alors qu'elles sont situées à l'intérieur d'un obstacle solide (par exemple en raison d'un pas de lancer de rayon qui traverse une arête fine). Une telle fuite fausse localement le gradient du SDF et aveugle le contrôleur, autorisant le drone à planifier une vitesse orientée vers l'obstacle ;
- **Les cellules fantômes (*Ghosts*)** : cellules marquées *Occupées* dans le vide à une distance supérieure à un seuil de tolérance ($\epsilon_{\text{ghost}} = 0,15$ m) de toute surface réelle, générant des faux positifs qui entravent inutilement les trajectoires.

Pour certifier la chaîne de perception [LiDAR → Projection → Grille], un outil d’audit automatisé (`check_map_leaks.py`) a été développé. À l’issue des expérimentations, cet outil compare chaque cellule observée de la grille locale d’un drone avec les polygones de la vérité terrain via des requêtes d’inclusion géométrique (`Polygon.contains`). Il quantifie le volume de cellules fuyantes et fantômes, identifie les obstacles critiques, et génère des cartes de diagnostic visuelles superposant les contours réels et les anomalies de classification. Ce banc de contrôle a permis d’éliminer les artefacts d’échantillonnage et de garantir que la perception locale reflète fidèlement la physique du monde simulé.

La représentation spatiale étant établie, la sous-section suivante expose le cadencement temporel garantissant la reproductibilité stricte et la cohérence de l’exécution multi-robots.

1.3 Gestion séquentielle et synchrone des pas de temps

La validité métrologique et la fidélité physique des modèles présentés reposent sur une gestion rigoureuse de la dynamique temporelle. Dans une simulation multi-robots, le temps physique simulé doit être strictement découplé de l’horloge réelle de la machine hôte : si les robots s’exécutaient de manière asynchrone au fil de l’eau, le temps accordé à chaque robot dépendrait de la durée de ses calculs et de la charge du système d’exploitation. Cette dérive détruirait le déterminisme du système et compromettrait la reproductibilité stricte des campagnes expérimentales.

1.3.1 Ordonnancement pas-à-pas synchrone (*Lockstep*)

Pour s’affranchir des aléas de l’ordonnanceur de l’OS hôte, le moteur de simulation met en œuvre un ordonnancement pas-à-pas synchrone (*lockstep synchronization*). Le temps continu est discrétisé en pas réguliers d’indice $k \in \mathbb{N}$ associés à un incrément temporel constant Δt (par défaut $\Delta t = 0,1$ s, correspondant à une fréquence de contrôle $f_s = 10$ Hz) :

$$t_k = k \cdot \Delta t, \quad k \in \mathbb{N} \quad (8)$$

Le principe fondamental de cette synchronisation par barrière temporelle impose qu’aucun robot ne soit autorisé à calculer son pas $k + 1$ tant que l’ensemble des robots de la flotte et le module de dynamique globale n’ont pas complété et validé l’intégralité de leur cycle d’exécution au pas k .

1.3.2 Isolation des processus et synchronisation par tubes IPC

L’intégration du langage d’essaim *Buzz* via sa bibliothèque d’exécution en C pose une contrainte technique fondamentale. Dans son implémentation d’origine, la machine virtuelle *Buzz* recourt à des variables d’état globales et statiques. Par conséquent, instancier plusieurs machines virtuelles au sein de différents fils d’exécution (*threads*) d’un même processus provoquerait des conflits d’accès mémoire et des corruptions d’état critiques.

Pour lever ce verrou d’ingénierie, chaque drone est isolé au sein d’un **processus système indépendant** (*DroneAgent* via le module standard Python `multiprocessing`).

Cette architecture répond à un double objectif :

1. **Isolation mémoire stricte** : chaque processus dispose de son propre espace d'adressage, garantissant l'intégrité de la machine virtuelle Buzz associée ;
2. **Exploitation du parallélisme multicœur** : les calculs lourds de chaque drone (rayons LiDAR, transformée de distance SDF, filtrage CBF) sont exécutés simultanément sur les différents cœurs CPU de la machine hôte, réduisant considérablement la durée globale de simulation.

Afin de préserver le déterminisme et le synchronisme temporel entre ces processus parallèles, deux couches de communication distinctes coexistent :

- **L'horloge et la barrière temporelle (Niveau Simulation)** : gérées par des tubes de communication inter-processus bidirectionnels (*IPC pipes*) reliant le coordinateur central (`lockstep_simulation.py`) à chaque processus drone ;
- **Le réseau d'essaim et le partage d'état distribué (Niveau Mission)** : assurés par l'intergiciel de communication décentralisée *Zenoh* via la passerelle d'interopérabilité *PyBuzz*. Les paquets émis au cours du pas k ne sont délivrés aux pairs qu'au pas $k + 1$, éliminant tout artefact de causalité temporelle induit par des durées d'exécution inégales entre processus.

1.3.3 Déroulement du cycle élémentaire de simulation

À chaque pas temporel discret k , le moteur déroule une séquence rigoureusement articulée entre la coordination globale et les calculs bord décentralisés :

1. **Propagation de l'environnement global (Maître)** : mise à jour de la vérité terrain de la cible terrestre mobile selon son profil cinématique continu ;
2. **Diffusion de l'ordre d'exécution (STEP)** : le coordinateur envoie l'instruction de calcul à travers les tubes IPC vers tous les drones en parallèle ;
3. **Acquisition sensorielle et cartographie locale (Drones)** : chaque robot calcule les intersections de ses rayons LiDAR 2D avec l'index spatial *R-tree*, met à jour sa grille d'occupation $\mathcal{M}_i$, en déduit la transformée de distance euclidienne (EDT/SDF locale), et met à jour la cinématique de la cible via son état de vérité terrain ;
4. **Planification et coordination décentralisée (Drones)** : sélection locale des objectifs de vol (exploration par échantillonnage ou repli énergétique sur base), exécution du pas d'évaluation de la VM Buzz et mise à jour des rôles par table partagée distribuée ;
5. **Filtrage de sécurité cinématique CBF (Drones)** : projection de la consigne nominale $\mathbf{u}_{\text{nom}}$ dans le demi-espace admissible de non-collision avec les obstacles via le solveur CBF (`solve_obstacle_cbf`) ;
6. **Intégration d'état, journalisation et barrière (ACK)** : application de la cinématique discrète, décrémentation de la jauge de batterie, contrôle géométrique de non-collision, journalisation télémétrique et émission de l'acquittement au coordinateur.

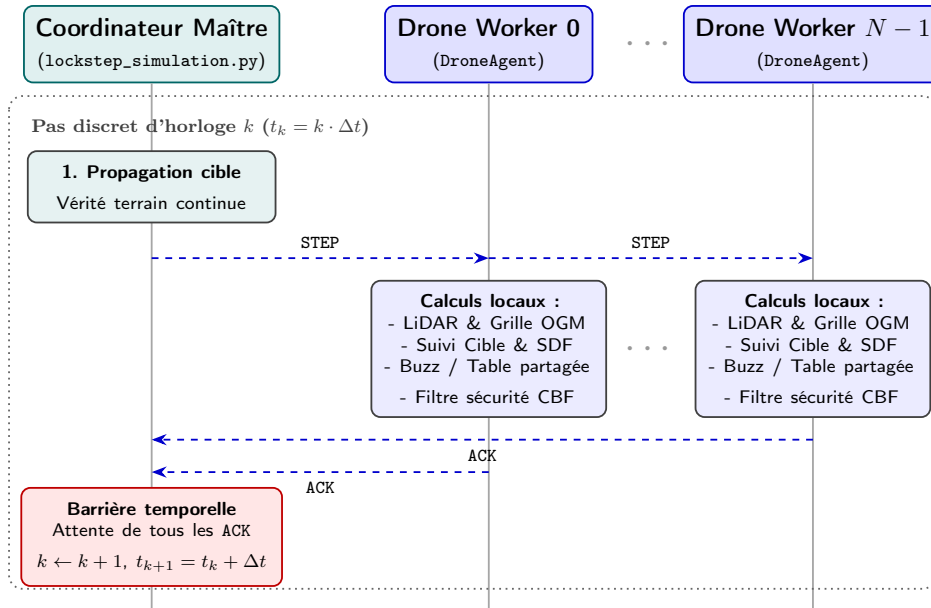


FIGURE 2 – Chronogramme de synchronisation synchrone (*lockstep*) par barrière IPC. Les processus drones exécutent leurs phases de perception, planification et contrôle en parallèle sur les cœurs CPU, tandis que l’avancement temporel global reste strictement cadencé par le processus maître coordinateur.

1.3.4 Régimes d’exécution, découplage temporel et contrainte du temps réel

Cette séparation stricte entre le temps simulé t_k et le temps d’horloge CPU machine (*wall-clock time*) apporte un bénéfice architectural déterminant face à la charge algorithmique élevée du système :

- **Charge computationnelle et exécution sans interface graphique (*headless*)** : dans la version actuelle du projet, l’empilement des traitements bord à bord à chaque cycle (lancer de 360 rayons LiDAR, transformée de distance euclidienne EDT/SDF sur grille discrète, expansion du graphe local et filtrage de sécurité CBF) s’avère particulièrement coûteux en temps processeur. Même exécuté sans affichage graphique (mode *headless*, éliminant tout surcoût de rendu pour évaluer le débit de calcul pur), le temps de calcul CPU moyen par pas τ_{calcul} oscille autour de 300 à 350 ms pour une flotte de 3 drones. Le moteur opère ainsi à environ un tiers de la vitesse nominale (le temps de calcul CPU moyen étant environ trois fois supérieur au pas de discrétisation physique $\Delta t = 100$ ms) ;
- **Incompatibilité provisoire avec le temps réel** : le système ne permet donc pas une exécution à la cadence nominale du temps réel (condition $\tau_{\text{calcul}} \leq \Delta t$) dans l’état actuel de son implémentation Python. Si une boucle asynchrone conventionnelle avait été employée, cette incapacité à respecter l’échéance temporelle ($\tau_{\text{calcul}} > \Delta t$) aurait provoqué des retards d’échantillonnage majeurs, des dépassements de pas d’intégration et une instabilité critique des barrières de sécurité CBF ;
- **Rôle protecteur de l’ordonnancement synchrone** : c’est précisément dans ce contexte de forte charge que l’architecture *lockstep* prend tout son sens. En suspendant l’avancement du temps simulé tant que la barrière d’acquiescement ACK

n'est pas validée par tous les cœurs, le moteur garantit une fidélité mathématique parfaite et une reproductibilité absolue des trajectoires, indépendamment du temps physique requis par la machine hôte pour résoudre les algorithmes.

Grâce à cet ordonnancement synchrone et à l'isolation étanche des environnements d'exécution, la simulation reproduit rigoureusement la dynamique d'une flotte décentralisée sans souffrir des distorsions liées au système hôte. Pour articuler l'ensemble de ces couches fonctionnelles au sein d'une base de code pérenne, la sous-section suivante expose l'architecture logicielle globale du simulateur ainsi que sa démarche d'assurance qualité.

1.4 Architecture logicielle modulaire et assurance qualité

La maintenabilité, la modularité et l'extensibilité du simulateur reposent sur une architecture logicielle découplée, s'appuyant sur des patrons de conception éprouvés et une séparation stricte des responsabilités entre composants.

1.4.1 Organisation modulaire des composants en couches

L'architecture globale du simulateur est structurée en quatre couches fonctionnelles interconnectées :

- **Couche Matérielle et Abstraction (platform)** : encapsule les grandeurs physiques du drone via la classe `DronePlatform` (patron Façade / HAL). Elle intègre la cinématique de vol, les capteurs et le modèle de batterie (`BatteryModel` régissant la décharge en vol à puissance $P_{\text{discharge}}$ et la recharge sur station à puissance P_{charge}), facilitant le portage futur vers des quadrirotors réels ;
- **Couche Perception et Cartographie (perception)** : centralise la mise à jour de la grille d'occupation ternaire (`OccupancyGridMap`) par politique optimiste via le patron Stratégie (`MapUpdateStrategy`, alternant Bresenham et tampon de profondeur vectorisé), la simulation LiDAR omnidirectionnelle (`LidarSimulator`, 360 rayons), la génération de champ de distance (`SDFGenerator`) et le suivi de la cible (`TargetBelief`) ;
- **Couche Décisionnelle et Sécurité (planning, controller)** : associe le planificateur sur graphe local (`Planner`, `LocalGraph`) évaluant les trajectoires par scoring multi-objectifs, le repli global Fringe-Retrieved A* (`FRAStar`), la passerelle Cython vers Buzz (`BuzzInterface`) et le filtre de sécurité cinématique (`cbf_solver.py` résolu via OSQP) ;
- **Couche Orchestration et Télémétrie (simulation, tools)** : cadence l'exécution synchrone (`lockstep_simulation.py`) reliant par tubes IPC les processus drones (`DroneAgent`), pilote l'interface graphique (`visualizer`) et enregistre une double métrologie (`TelemetryLogger` en CSV et `StepTracer` événementiel en JSONL).

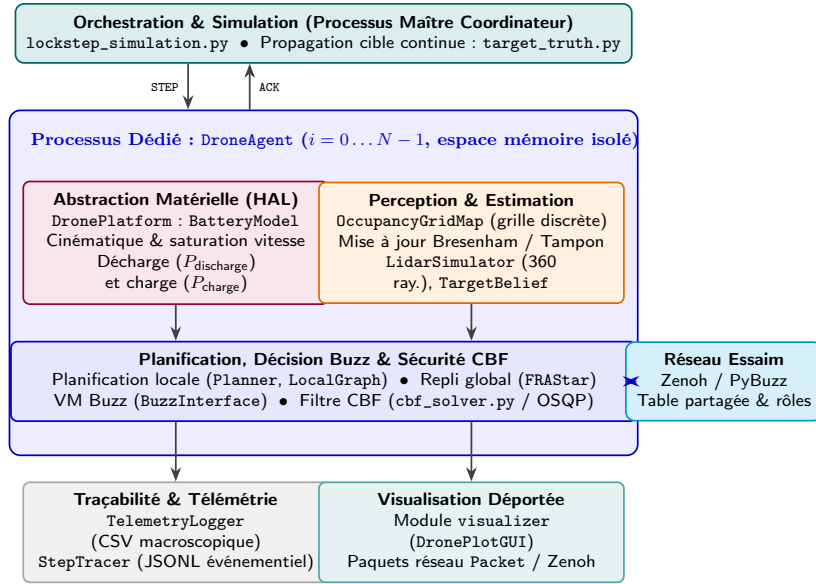


FIGURE 3 – Architecture logicielle modulaire en couches et flux d’orchestration : isolation mémoire des robots par processus, couches fonctionnelles internes, télémétrie et synchronisation d’état distribuée.

1.4.2 Démarche d’assurance qualité et analyse critique de la validation logicielle

Le développement du simulateur a suivi une méthodologie rigoureuse d’ingénierie logicielle garantissant l’exactitude des implémentations et la reproductibilité expérimentale :

- **Suite de tests automatisés** : 33 scripts sous *pytest* totalisant 222 tests unitaires et d’intégration validant la décroissance d’énergie, l’invariance de sécurité du filtre CBF et la conformité stricte JSON/Buzz (`test_config_consistency.py`) ;
- **Couverture multi-processus et typage statique strict** : traçabilité multi-processus via *Coverage.py*, couplée au respect strict du standard PEP 484 vérifié sans erreur sous *Pyright* et *Pylint* (note de 10.00/10) ;
- **Exécution non graphique** : découplage calcul/rendu pour exécuter des campagnes massives en arrière-plan (`run_mission_headless.py`, `run_sweep.sh`).

Enseignements de l’audit de code et limites des métriques de surface. Une contribution méthodologique importante réside dans le constat qu’une conformité stylistique et une couverture de code élevées ne suffisent pas à garantir la validité physique d’un système robotique. Lors des audits approfondis, trois anomalies sémantiques subtiles ont été levées :

1. L’absence initiale du facteur de fréquence dans la dérivée discrète de la barrière énergétique Buzz, sous-estimant la contrainte de retour d’un ordre de grandeur tout en validant les tests unitaires ;
2. L’apparition de cas dégénérés dans le programme quadratique lorsque la distance au waypoint devenait inférieure au millimètre, provoquant une infaisabilité numérique artificielle ;

3. Des phénomènes d'intersections diagonales non désirées (*corner-cutting*) dans la reconstruction incrémentale des chemins de repli global.

Ces retours d'expérience démontrent l'importance capitale des bancs de diagnostic fonctionnels croisés (audit des fuites de cartes, analyse télémétrique pas-à-pas) face aux limites des tests purement structurels.

Synthèse de la section et transition. Cette première section a permis d'établir les fondations physiques, cinématiques et logicielles du système à travers un simulateur multi-robots synchrone et déterministe sous observabilité partielle. Disposant d'un modèle cinématique réaliste, d'une cartographie dynamique avec calcul de gradient spatial et d'un ordonnancement robuste, l'étude peut désormais aborder la problématique algorithmique centrale : comment orchestrer la coordination décentralisée d'une flotte pour assurer une mission persistante tout en garantissant formellement la sécurité du vol et la viabilité énergétique ? Cette question fait l'objet de la section 2.

2 Planification de missions persistantes dans un environnement partiellement observable

Tandis que la première partie de ce rapport a établi le socle expérimental en modélisant la cinématique des quadrirotors, la dynamique des accumulateurs, la perception discrète par grilles d’occupation et champs de distance signée, ainsi que l’infrastructure de simulation synchrone en pas cadencés (*lockstep*), cette seconde partie est consacrée à la couche décisionnelle et au contrôle sécurisé de la flotte.

La pérennité d’une mission de surveillance aérienne opérée par des quadrirotors à autonomie énergétique limitée exige une articulation étroite entre décision collective distribuée, anticipation spatio-temporelle de trajectoire sous observabilité partielle et garanties formelles d’invariance sécuritaire.

Le présent chapitre expose la chaîne algorithmique conçue pour répondre à ce triple impératif :

- La sous-section 2.1 formalise la machine à états finis décentralisée et le protocole de relève régissant le cycle de vie de la mission persistante ;
- La sous-section 2.2 développe la stratégie de planification hiérarchique à deux échelles, combinant un corridor macroscopique incrémental et un graphe aléatoire local orienté vers l’exploration active des frontières ;
- La sous-section 2.3 détaille la synthèse des fonctions barrières de contrôle (CBF), articulant un filtre cinématique réactif d’évitement d’obstacles (OGM-CBF) et des barrières d’invariance d’état pour la viabilité énergétique et la relève ;
- Enfin, la sous-section 2.4 présente les résultats expérimentaux du balayage paramétrique sur le prototype préliminaire, conduit une analyse critique approfondie des écarts avec l’architecture cible (notamment sur la sécurité énergétique et inter-drones), et dégage la feuille de route technique pour son achèvement.

2.1 Architecture décentralisée et cycle de vie de la mission persistante

La surveillance pérenne d’une zone ou d’une cible mobile au moyen de quadrirotors est intrinsèquement limitée par l’autonomie des batteries. Pour maintenir une couverture continue sans rupture d’observation, les drones doivent opérer des relèves mutuelles en vol. Dans un contexte opérationnel soumis à des contraintes de communication, confier cette coordination à une station sol ou à un maître centralisé introduirait un point unique de défaillance (*single point of failure*) préjudiciable à la résilience du système. La coordination de la flotte est donc entièrement décentralisée, chaque robot exécutant localement la même logique de contrôle sous la machine virtuelle *Buzz* [5].

Partage d’état et protocole de passage de témoin. Pour synchroniser leurs décisions sans serveur central, les robots s’appuient sur le mécanisme de table partagée distribuée (désigné sous l’appellation métaphorique de « stigmergie virtuelle » dans les primitives

de *Buzz*). Chaque drone y publie périodiquement son état courant (identifiant, rôle opérationnel, position et niveau de batterie).

Sur la base de ces informations partagées, la flotte implémente un protocole déterministe de « passage de témoin » (*baton passing*) afin de prévenir tout conflit lors de la relève : le drone de mission actif en vol évalue périodiquement l'état des aéronefs disponibles sur la station de recharge. Dès qu'un remplacement devient nécessaire, il élit le candidat en attente (READY) disposant de la plus grande réserve d'énergie et diffuse une requête unilatérale `pursuer_request`. Le candidat désigné arme alors localement son drapeau de poursuite (PURSUER_BATON_FLAG), garantissant qu'un unique drone décolle pour effectuer la relève.

Cycle de vie et machine à états finis (FSM) distribuée. La dynamique opérationnelle de chaque aéronef est gouvernée par une machine à états finis distribuée (*Finite State Machine* – FSM), dont la topologie adapte et enrichit le formalisme de relève coopérative introduit par Fouad, Zhao *et al.* [1]. La figure 4 détaille l'agencement des états et les conditions de transition régissant la flotte.

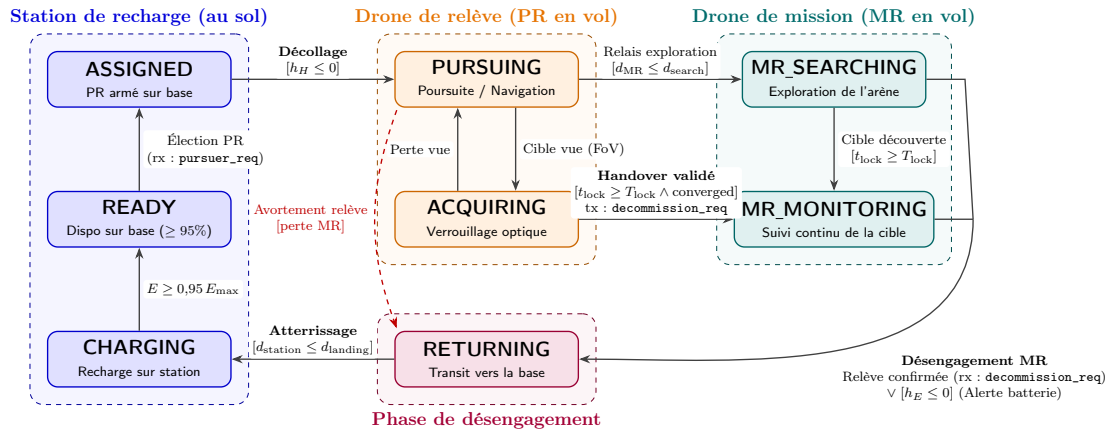


FIGURE 4 – Machine à états finis (FSM) distribuée embarquée sur chaque drone : cycle nominal de persistance de mission, mécanismes de relève coordonnés par table partagée et transitions d'avortement sécurisées.

Comme l'illustre la figure 4, le cycle de persistance de la mission s'articule autour de quatre phases fonctionnelles interconnectées :

1. **Phase 1 – Station de recharge (au sol)** : l'aéronef se recharge sur sa base (CHARGING) jusqu'à recouvrer 95 % de sa capacité nominale ($E \geq 0,95 E_{\max}$), basculant en attente active (READY). À réception du message `pursuer_request`, le drone élu s'arme pour la relève (ASSIGNED) et précalcule son corridor de décollage ;
2. **Phase 2 – Drone de relève (Pursuer Robot – PR en vol)** : l'ordre de décollage vers la zone d'opération (PURSUING) est déclenché de façon déterministe dès que la barrière de relève franchit son seuil critique ($h_H \leq 0$, section 2.3). À l'approche de la zone, dès que la cible se situe à portée et en visibilité directe (*Line-of-Sight* – LoS non occluse), le drone enclenche la phase d'acquisition (ACQUIRING) ;

3. **Phase 3 – Drone de mission (*Mission Robot* – MR en vol)** : le transfert officiel d'autorité (*handover*) est validé lorsque la cible est maintenue verrouillée sous visibilité continue pendant une durée minimale T_{lock} et que la convergence du filtre d'état est confirmée. Le remplaçant bascule alors en surveillance active (MR_MONITORING) et notifie l'ancien drone par diffusion du message `decommission_request`. En cas d'initialisation sans cible repérée, le drone patrouille en mode recherche (MR_SEARCHING) jusqu'à la première détection directe de la cible ;
4. **Phase 4 – Désengagement et repli sécurisé** : dès réception de l'ordre de décommissionnement, ou lors d'une alerte énergétique critique ($h_E \leq 0$), l'ancien drone de mission engage un vol de retour autonome vers la station (RETURNING). Dès confirmation de l'atterrissage ($d_{\text{station}} \leq d_{\text{landing}}$), le drone réintègre l'état de recharge initiale CHARGING, refermant ainsi le cycle de persistance.

Cette structure d'états discrets garantit l'ordonnancement global des rôles au sein de la flotte. Néanmoins, l'exécution physique de chaque phase exige de guider les drones à travers un environnement partiellement cartographié et parsemé d'obstacles imprévus. La sous-section 2.2 suivante présente le planificateur de trajectoire spatio-temporel développé pour concilier progression opérationnelle, évitement d'obstacles et exploration active de l'inconnu.

2.2 Planification spatio-temporelle sur graphe local et repli global

Tandis que la machine à états finis décentralisée régit les transitions discrètes de l'essaim (section 2.1), la navigation autonome sous observabilité partielle requiert de générer en continu des trajectoires admissibles conciliant ralliement d'objectif, exploration opportuniste de raccourcis et évitement préventif des impasses.

2.2.1 Architecture hiérarchique à deux échelles

Dans une carte partiellement explorée, un planificateur global purement guidé par l'objectif (*goal-directed*) tel que l'algorithme incrémental *Fringe-Retrieving A** (FRA*) [11] planifie exclusivement sur les zones certifiées libres, s'enfermant fréquemment dans de longs détours de contournement. À l'inverse, une exploration purement locale risquerait de piéger l'aéronef dans des culs-de-sac.

Pour résoudre ce dilemme, une **architecture hiérarchique à deux échelles** est mise en œuvre :

1. **À l'échelle macroscopique (corridor FRA*)** : un itinéraire de référence global relie en continu le drone à sa destination en contournant les obstacles connus sur la grille globale $\mathcal{M}$, fournissant à tout point $\mathbf{p}$ une distance géodésique résiduelle $d_{\text{FRA}^*}(\mathbf{p}, \mathbf{p}_{\text{goal}})$;
2. **À l'échelle microscopique (graphe d'accessibilité local RRG)** : un graphe spatio-temporel aléatoire (*Rapidly-exploring Random Graph* – RRG) [12, 13] est

déployé autour du drone dans une fenêtre locale de $40 \times 40 \text{ m}^2$. En sondant opportunément les frontières de l'inconnu, ce planificateur découvre des brèches directes tout en garantissant la progression globale.

2.2.2 Graphe local RRG et échantillonnage probabiliste biaisé

Pour concentrer les degrés de liberté là où l'utilité opérationnelle est maximale, l'échantillonnage des $N = 60$ sommets du graphe local repose sur un **tirage probabiliste biaisé à quatre composantes** (figure 5) : 60 % de tirages uniformes (couverture locale omnidirectionnelle), 20 % dans un cône polaire orienté vers la cible ($\sigma_\theta = 0,4 \text{ rad}$), 10 % vers les cellules de frontière (cellules libres contiguës à l'inconnu, stimulant la découverte), et 10 % le long du corridor global FRA* ($\sigma_{\text{line}} = 1,0 \text{ m}$, progression à long terme).

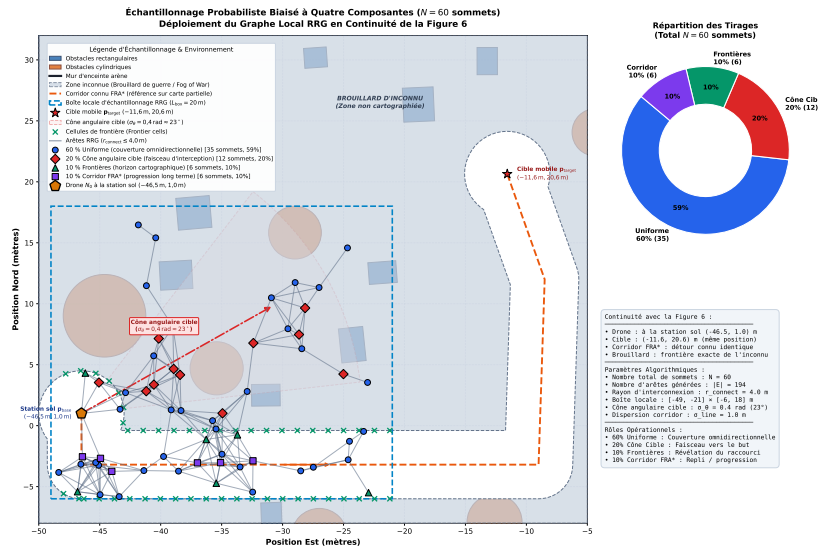


FIGURE 5 – Échantillonnage probabiliste biaisé à quatre composantes pour la génération du graphe local RRG ($N = 60$ sommets, portée d'arête $r_{\text{edge}} \leq 4,0 \text{ m}$) : couverture omnidirectionnelle (60 %, disques bleus), cône polaire d'interception de la cible (20 %, losanges rouges), exploration active des frontières (10 %, triangles verts) et progression le long du corridor FRA* (10 %, carrés violets).

Chaque sommet échantillonné subit une extension cinématique (*steer*) bornée à $r_{\text{edge}} = 4,0 \text{ m}$ du nœud le plus proche. Les sommets $v_k = (\mathbf{p}_k, \tau_k) \in \mathbb{R}^2 \times \mathbb{R}^+$ propagent le temps d'arrivée prévisionnel selon la vitesse nominale : $\tau_{k+1} = \tau_k + \|\mathbf{p}_{k+1} - \mathbf{p}_k\|/v_{\text{cruise}}$. Une arête (v_i, v_j) n'est validée que si son corridor tubulaire de sécurité est entièrement libre : $\text{Tube}(\mathbf{p}_i, \mathbf{p}_j, r_{\text{safe}}) \subset \mathcal{M}_{\text{free}}$.

2.2.3 Formulation des gains de mission et modélisation du risque

L'arbre des plus courts chemins depuis la racine est extrait par l'algorithme de Dijkstra. Chaque trajectoire candidate $\pi = (v_0, \dots, v_K)$ est évaluée selon des critères quantitatifs complémentaires :

- **Gain d'information cartographique active** (G_{info}) : union ensembliste des cellules inconnues $\mathcal{U}_{\text{visible}}(v_k)$ balayées par un lancer virtuel de 36 rayons (portée 4 m) sans obstruction :

$$G_{\text{info}}(\pi) = \text{card} \left(\bigcup_{v_k \in \pi} \mathcal{U}_{\text{visible}}(v_k) \right) \quad (9)$$

Inspirée des travaux sur l'acquisition active d'information en robotique mobile [14, 15], cette formulation par union ensembliste proscriit tout surcomptage artificiel qui favoriserait les trajectoires oscillantes ;

- **Gain de suivi continu de la cible** (G_{track}) : intégration temporelle pondérée de la proximité spatiale et de la visibilité directe en ligne de vue (*Line-of-Sight* – LoS) :

$$G_{\text{track}}(\pi) = \frac{1}{\Delta\tau} \sum_{k=0}^{K-1} (\tau_{k+1} - \tau_k) \cdot \mathbb{I}_{\text{LoS}}(v_k) \cdot \exp \left(-\frac{1}{2} \left(\frac{\|\mathbf{p}_k - \mathbf{p}_T\|}{\sigma_{\text{dist}}} \right)^2 \right) \quad (10)$$

où $\sigma_{\text{dist}} = 5,0$ m et $\mathbb{I}_{\text{LoS}}(v_k) = 1$ si la cible est dans le champ visuel sans occlusion ;

- **Progression vers l'objectif** (G_{goal}) : réduction géodésique différentielle le long du corridor FRA* :

$$G_{\text{goal}}(\pi) = \max \left(0, d_{\text{FRA}^*}(\mathbf{p}_0, \mathbf{p}_{\text{goal}}) - d_{\text{FRA}^*}(\mathbf{p}_K, \mathbf{p}_{\text{goal}}) \right) \quad (11)$$

- **Risque de déviation** (R_{dev}) et **longueur effective** (L_{eff}) : longer des zones inexplorées expose l'aéronef à découvrir tardivement des impasses imposant de longs détours. En intégrant la densité locale de risque $r_{\text{dev}}(v)$ (définie sur le voisinage de rayon 2 m où les cellules occupées pèsent 1,0 et les cellules inconnues $\beta_U = 0,5$), la longueur effective majorante s'exprime par :

$$L_{\text{eff}}(\pi) = L(\pi) + \kappa_R \cdot R_{\text{dev}}(\pi), \quad \text{avec } R_{\text{dev}}(\pi) = \sum_{e_k \in \pi} r_{\text{dev}}(v_k) \cdot \ell(e_k) \quad (12)$$

avec $\kappa_R = 1,5$. Cette longueur effective alimente directement la réserve énergétique de retour ;

- **Pénalité de courbure** (C_{smooth}) : somme des variations angulaires relatives entre segments consécutifs, pénalisant les zigzags cinématiquement défavorables.

2.2.4 Optimisation multi-objectifs, normalisation Min-Max et sélection

Pour agréger ces grandeurs hétérogènes, chaque critère est normalisé sans dimension via Min-Max : $\bar{\phi}_k(\pi) \in [0, 1]$. Le score composite universel s'exprime alors par :

$$J_i(\pi) = w_I \bar{G}_{\text{info}}(\pi) + w_G \bar{G}_{\text{goal}}(\pi) + \mathbb{I}_{\text{pursuit}}(i) \cdot w_T \bar{G}_{\text{track}}(\pi) - w_L \bar{L}_{\text{eff}}(\pi) - w_R \bar{R}_{\text{dev}}(\pi) - w_S \bar{C}_{\text{smooth}}(\pi) \quad (13)$$

Le chemin optimal retenu $\pi^* = \arg \max_{\pi \in \Pi} J_i(\pi)$ fournit immédiatement le premier waypoint converti en consigne de vitesse nominale $\mathbf{u}_{\text{nom}}$, transmise à l'étage de régulation sécuritaire (section 2.3).

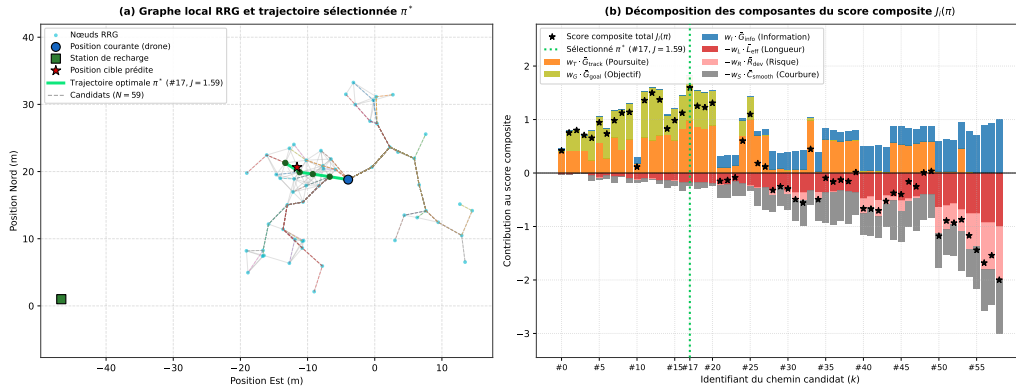


FIGURE 6 – Visualisation et diagnostic de la sélection de trajectoire sur graphe local RRG (outil `plot_planning_graph.py`). (a) Déploiement spatial du graphe, des 59 candidats admissibles et de la trajectoire optimale π^* sélectionnée. (b) Décomposition des critères du score composite $J_i(\pi)$, identifiant le candidat n°17 comme maximisant le compromis global ($J = 1,59$).

Un outil d'inspection et de télémessure graphique (`plot_planning_graph.py`) a été développé pour analyser ces arbitrages. La figure 6 illustre ce processus : la trajectoire optimale retenue (n°17) concilie une attraction élevée vers la cible ($w_G \bar{G}_{\text{goal}} + w_T \bar{G}_{\text{track}}$) tout en minimisant la longueur effective et le risque de déviation.

L'actualisation dynamique du corridor global de repli s'appuie sur l'algorithme incrémental FRA* [11], qui restreint la réoptimisation aux seules branches de l'arbre invalidées par les nouvelles observations LiDAR, réduisant drastiquement la charge CPU. Disposant de cette consigne nominale $\mathbf{u}_{\text{nom}}$, la sous-section suivante formalise les garanties de sécurité par fonctions barrières de contrôle (CBF).

2.3 Conception des fonctions barrières de contrôle (CBF)

Tandis que le planificateur spatio-temporel (section 2.2) synthétise les consignes de vitesse nominales $\mathbf{u}_{\text{nom}}$ guidées par les objectifs de mission, la sécurité critique du vol et la pérennité énergétique requièrent des garanties formelles d'invariance. L'architecture de contrôle s'appuie à ce titre sur le cadre théorique des *Control Barrier Functions* (CBF) formalisé par Ames *et al.* [3]. Dans notre approche, une distinction claire et modulaire est établie entre :

1. Un **filtre cinématique réactif d'évitement d'obstacles** (OGM-CBF), garantissant la non-collision avec les obstacles physiques par projection instantanée de la consigne de vitesse ;
2. Des **fonctions barrières discrètes d'invariance d'état** (h_E et h_H), régissant la viabilité énergétique de la flotte, la sélection des trajectoires admissibles et le déclenchement décentralisé des relèves.

2.3.1 Filtre cinématique réactif d'évitement d'obstacles (OGM-CBF)

Soit un ensemble de sécurité admissible vis-à-vis des obstacles physiques noté $\mathcal{C}_{\text{obs}} = \{\mathbf{p} \in \mathbb{R}^2 \mid h_{\text{obs}}(\mathbf{p}) \geq 0\}$. En modélisant le quadricoptère comme un système cinématique à intégrateur simple ($\dot{\mathbf{p}} = \mathbf{u}$), la fonction barrière présente un degré relatif de 1 par rapport à la commande en vitesse $\mathbf{u}$. Selon le théorème de Nagumo et les propriétés d'invariance directe des CBF [3], l'ensemble $\mathcal{C}_{\text{obs}}$ est rendu invariant sous l'action de commandes satisfaisant :

$$\dot{h}_{\text{obs}}(\mathbf{p}, \mathbf{u}) = \nabla h_{\text{obs}}(\mathbf{p})^\top \mathbf{u} \geq -\gamma_{\text{obs}} h_{\text{obs}}(\mathbf{p}) \quad (14)$$

où $\gamma_{\text{obs}} > 0$ est un gain linéaire de convergence réglant la raideur de répulsion aux frontières du domaine sûr.

Dans notre simulateur, ce filtre de sécurité est directement connecté à la perception locale via le champ de distance signée (SDF) introduit à la section 1.2. En chaque point $\mathbf{p}$, la distance euclidienne continue à l'obstacle le plus proche est notée $D(\mathbf{p})$ et son gradient spatial $\nabla D(\mathbf{p}) = [d_n, d_e]^\top$ indique la direction de fuite locale ($\|\nabla D(\mathbf{p})\| \approx 1$). La barrière d'obstacle s'écrit alors :

$$h_{\text{obs}}(\mathbf{p}) = D(\mathbf{p}) - r_{\text{safe}} \quad (15)$$

où $r_{\text{safe}} = r_{\text{drone}} + d_{\text{safe}}$ englobe le gabarit physique du drone et une marge de sécurité.

À chaque pas de commande ($f = 10$ Hz), le module de contrôle (`cbf_solver.py`) reçoit la consigne de vitesse nominale $\mathbf{u}_{\text{nom}}$ issue de l'optimisation sur le graphe local (section 2.2). Le filtre cinématique calcule la vitesse corrigée $\mathbf{u}^*$ projetée au plus près de la consigne dans le demi-espace admissible :

$$\mathbf{u}^* = \arg \min_{\mathbf{u} \in \mathbb{R}^2} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|^2 \quad (16)$$

sous l'unique contrainte barrière d'évitement d'obstacles :

$$\nabla D(\mathbf{p})^\top \mathbf{u} \geq -\gamma_{\text{obs}} (D(\mathbf{p}) - r_{\text{safe}}) \quad (17)$$

Sur le plan de l'implémentation (`solve_obstacle_cbf`), ce problème de projection orthogonale sur un demi-espace affine se formule sous la forme standard d'un programme quadratique minimal $\frac{1}{2} \mathbf{u}^\top P \mathbf{u} + \mathbf{q}^\top \mathbf{u}$ avec $P = 2I_2$, $\mathbf{q} = -2\mathbf{u}_{\text{nom}}$, et se résout via la bibliothèque `qpsovers` avec le solveur standard OSQP (*Operator Splitting Quadratic Program*). Puisque la contrainte définit un unique demi-espace dans le plan $\mathbb{R}^2$, le problème est strictement convexe et trivialement faisable (en cas de gradient quasi-nul $\|\nabla D\| < 10^{-9}$, la commande nominale est conservée sans altération), ce qui en fait un simple filtre de projection léger en bout de chaîne de guidage. Le bornage cinématique en vitesse admissible ($\|\mathbf{u}^*\| \leq v_{\text{max}}$) est appliqué immédiatement après par une saturation modulaire (`clamp_step_velocity` dans `flight.bzz`).

2.3.2 Barrière énergétique d'autonomie et de repli sécurisé

Contrairement aux obstacles physiques dont l'évitement requiert une déviation cinématique instantanée, la gestion de l'énergie repose sur une décision discrète de repli. Pour certifier qu'aucun aéronef ne tombe en panne de batterie en vol, la fonction barrière énergétique h_E compare l'état de charge résiduelle $E_i(t)$ à l'énergie de transit nécessaire pour regagner la base de recharge, conformément au formalisme d'invariance d'état énergétique formalisé par Fouad, Zhao *et al.* [1] et Fouad *et al.* [8] :

$$h_E(t) = E_i(t) - E_{\min} - \left(P_{\text{discharge}} \cdot \frac{L_{\text{return}}(\mathbf{p}_i, \mathbf{p}_{\text{base}})}{v_{\text{cruise}}} + E_{\text{reserve}} \right) \quad (18)$$

où $P_{\text{discharge}}$ est la puissance moyenne consommée en vol, v_{cruise} la vitesse de croisière nominale, L_{return} la longueur effective du chemin de retour station extrait par l'algorithme incrémental FRA* (section 2.2), $E_{\min}$ la réserve terminale d'atterrissage, et E_{reserve} une marge couvrant les incertitudes de modèle et les détours d'évitement.

Dans notre architecture logicielle, $h_E(t)$ agit à deux niveaux clés :

- **Transition FSM de sécurité** (`flight.bzz`) : dès que la condition barrière franchit son seuil critique :

$$h_E(t) \leq 0 \quad (19)$$

le drone de mission interrompt immédiatement sa tâche de surveillance et engage une transition prioritaire vers l'état de vol de retour (`RETURNING`) ;

- **Filtrage des trajectoires admissibles** (`planner.py`) : lors de la planification sur graphe local (section 2.2), la fonction `_is_node_energy_eligible` élimine tout nœud dont la consommation prévisionnelle compromettrait la satisfaction stricte de $h_E \geq 0$ au retour, garantissant l'invariance prédictive du domaine de viabilité.

2.3.3 Barrière de relève coopérative décentralisée

Pour maintenir la persistance indéfinie de la surveillance sans rupture de couverture, la flotte synchronise le départ du drone de relève (PR) au moyen d'une seconde fonction barrière coopérative notée h_H . Calculée au sol par le drone armé (`ASSIGNED`), cette barrière évalue si l'autonomie restante du drone en mission ($h_{E,\text{MR}}$, partagée périodiquement via la table distribuée) suffit à couvrir le temps d'approche du remplaçant :

$$h_H(t) = h_{E,\text{MR}}(t) - \eta - \hat{E}_H^{\text{risk}} \quad (20)$$

où $\eta > 0$ est une marge temporelle de sécurité assurant le chevauchement visuel de la passation, et $\hat{E}_H^{\text{risk}}$ représente le coût énergétique prévisionnel complet du vol d'interception du PR :

$$\hat{E}_H^{\text{risk}} = \frac{P_{\text{discharge}}}{v_{\text{cruise}}} \left(L_H(t) + \kappa_H R_H(t) \right) + E_{\text{acq}} + E_{\text{reserve}} \quad (21)$$

avec $L_H(t)$ la distance au point de rendez-vous futur estimé sur le corridor global FRA*, $R_H(t)$ le risque de détour associé le long du corridor (eq. 12), et E_{acq} l'énergie allouée à la

phase d'acquisition et à la stabilisation du verrouillage sur la cible pendant la durée de confirmation T_{lock} .

Puisque le terme $\hat{E}_H^{\text{risk}}$ quantifie de manière prospective le temps et l'énergie du trajet aller du PR, la barrière $h_H(t)$ est intrinsèquement prédictive par conception. L'ordre de décollage du robot de relève (T3: ASSIGNED $\rightarrow$ PURSUING) est ainsi déclenché de façon directe et instantanée dès que la marge de viabilité s'annule dans `handover.bzz` :

$$h_H(t) \leq 0 \quad (22)$$

Ce mécanisme est rigoureusement **déterministe et sans hystérésis** : une fois le seuil $h_H \leq 0$ franchi, le PR s'engage définitivement dans sa mission de ralliement, éliminant tout risque de battements ou d'oscillations parasites entre les états armé et en vol.

2.4 Campagnes expérimentales, analyse critique et feuille de route

Afin d'évaluer le comportement de la flotte multi-robots, la sensibilité des critères de mission et la dynamique de relève sous observabilité partielle, une série de simulations a été conduite dans l'environnement développé.

Il convient d'explicitier le périmètre de cette évaluation. Compte tenu de l'envergure du développement de la plateforme de simulation (moteur synchrone Python/Buzz en pas cadencé, perception LiDAR, cartographie OGM et champs SDF, isolation multi-processus et outils de télémétrie), le calendrier du stage a été prioritairement alloué à l'achèvement et à la certification de cette infrastructure socle (section 1). Dès lors, les essais présentés ci-après ont été conduits sur une **version intermédiaire du prototype** : les briques cinématiques et le planificateur RRG sont pleinement opérationnels, mais le couplage fermé prédictif de la barrière énergétique h_E avec FRA* et l'évitement inter-drones réciproque au niveau du filtre réactif CBF n'ont pas pu être totalement finalisés. Ces expérimentations préliminaires permettent de diagnostiquer précisément le comportement du système et d'apporter la preuve empirique du rôle indispensable des barrières formelles.

2.4.1 Protocole expérimental et métriques d'évaluation

Les expérimentations reprennent le cadre nominal établi dans la section 1 (flotte de $N = 3$ drones, arène de $25 \text{ m} \times 25 \text{ m}$, station à double chargeur et cible mobile à $v_{\text{target}} \approx 0,8 \text{ m/s}$). L'environnement intègre des obstacles concaves et des piliers générant de fortes occlusions afin de mettre à l'épreuve l'exploration et le maintien de contact.

Quatre indicateurs quantitatifs sont mesurés :

- **Taux de couverture continue** (`fraction_observed`) : proportion du temps où la cible est visible par au moins un drone ;
- **Interruption maximale** (ΔT_{max}) : durée maximale consécutive sans observation lors des relèves ;
- **Marge inter-robots** ($\min d_{\text{drone}}$) : distance relative minimale (conforme si > 0 , négative en cas de violation de d_{safe}) ;

- **Marge énergétique résiduelle** ($\min E_{\text{batt}}$) : énergie minimale au retour (négative en cas de panne sèche en vol).

2.4.2 Résultats empiriques du balayage paramétrique

Un balayage systématique de 35 expérimentations a été conduit (`weight_sweep.py`) en faisant varier individuellement chaque pondération $w_k \in [0,2; 1,0]$ de la fonction de score (13). Le tableau 2 rassemble les configurations les plus caractéristiques.

TABLE 2 – Synthèse représentative du balayage paramétrique des poids de trajectoire sur le prototype intermédiaire (extraits caractéristiques issus de 35 simulations complètes sous observabilité partielle).

Régime opérationnel	Poids w_k	Couverture (%)	ΔT_{max} (s)	$\min d_{\text{drone}}$ (m)	$\min E_{\text{batt}}$ (J)	Bilan physique (Séparation / Batterie)
Configuration nominale	$w_{\bullet} = 1,0$	89,8 %	6,8	-0,36	+75,4	Conflit inter-drones
Poursuite agressive	$w_{\text{track}} = 1,0$	91,1 %	6,8	-0,22	+123,7	Pénétration marge d_{safe}
Poursuite distante	$w_{\text{track}} = 0,2$	91,8 %	6,8	-0,40	+22,5	Conflit d'interception
Longueur modérée	$w_{\text{length}} = 0,4$	57,1 %	31,8	+1,98	+96,1	Séparation & Énergie OK
Longueur excessive	$w_{\text{length}} = 0,6$	57,0 %	31,9	+0,75	-1070,0	Effet falaise : panne sèche
Relève coordonnée	$w_{\text{handover}} = 0,2$	85,7 %	6,8	+0,60	+169,7	Transition nominale fluide
Exploration frontières	$w_{\text{info}} = 0,6$	71,0 %	6,9	+4,51	+457,2	Découverte sans conflit

L'examen des journaux télémétriques met en évidence deux comportements critiques qui constituent autant de pistes d'investigation prioritaires :

1. **Sensibilité marquée à la pénalité de longueur** (w_{length}) : pour $w_{\text{length}} \leq 0,4$, les drones concluent leur mission avec un solde énergétique positif (+96,1 J). En revanche, dès que $w_{\text{length}} \geq 0,6$, la télémétrie enregistre un effondrement de la jauge ($\min E_{\text{batt}} = -1070$ J, traduisant une simulation prolongée au-delà de l'épuisement théorique). Ce comportement suggère une difficulté du planificateur local à concilier raccourcissement des trajectoires et contournement opportun des zones encombrées, voire d'éventuels blocages locaux en cours de vol. Ces résultats automatisés constituent un premier diagnostic qui demandera une confirmation par relecture visuelle pas-à-pas ;
2. **Excursions de proximité en régime de suivi soutenu** : les configurations favorisant fortement la poursuite ($w_{\text{track}} \geq 0,6$) affichent une couverture élevée (≈ 91 %) mais s'accompagnent de marges de séparation négatives ($\min d_{\text{drone}} < 0$, atteignant -0,40 m), signalant un franchissement de l'enveloppe de confort d_{safe} . Même si l'absence de contraintes inter-robots dans le filtre réactif favorise de telles proximités lors de convergences de robots, ces occurrences doivent être minutieusement réanalysées pour isoler la part respective de la cinématique des drones et de potentiels biais de discrétisation du simulateur.

2.4.3 Analyse critique des écarts et feuille de route technique

Ces résultats illustrent avec netteté les limites structurelles d'un réglage heuristique des pondérations et confirment les deux chantiers restant à finaliser pour atteindre l'architecture

de sûreté formelle :

1. **Couplage fermé FRA* – barrière énergétique h_E** : Actuellement, le repli énergétique repose sur une transition FSM réactive. Pour éliminer les pannes de batterie observées, le calcul incrémental de L_{return} par FRA* doit être couplé en boucle fermée à chaque pas de planification pour invalider prédictivement tout nœud violant $h_E \geq 0$;
2. **Formulation d’un filtre CBF multi-robots réciproque** : Le filtre réactif OGM-CBF traite l’évitement des obstacles statiques cartographiés. Il devra être enrichi de contraintes barrières inter-drones réciproques fondées sur l’échange décentralisé de trajectoires pour prévenir systématiquement tout rapprochement excessif ($\min d_{\text{drone}} > 0$).

Cette confrontation lucide démontre la rigueur scientifique du diagnostic et fournit une feuille de route claire pour le déploiement sur système physique.

3 Conclusion

Ce stage réalisé au Laboratoire de Vision et Systèmes Numériques (LVSN) de l’Université Laval a permis de concevoir un simulateur multi-robots dédié à l’étude des environnements partiellement observables, d’implémenter un planificateur local sur graphe RRG instrumenté pour l’inspection multi-objectifs, et de formaliser une architecture rigoureuse de sécurité critique par fonctions barrières de contrôle (CBF) et repli énergétique.

À court terme, la priorité technique consistera à finaliser l’interconnexion logicielle en boucle fermée entre l’algorithme incrémental FRA* et le filtrage d’admissibilité énergétique h_E , tout en intégrant des contraintes barrières de séparation inter-drones réciproques au sein du filtre réactif de sécurité CBF, comblant ainsi les limites observées lors des campagnes préliminaires. À plus long terme, ces travaux ouvrent la voie à la prise en compte d’obstacles mobiles hautement dynamiques, à l’extension vers des CBF d’ordre supérieur (HOCBF) pour des modèles dynamiques non linéaires, ainsi qu’au déploiement expérimental sur une flotte de robots physiques.

Références

- [1] H. Fouad, H. Zhao, and G. Beltrame, “Persistent and reliable task execution in distributed robot teams : A CBF-based energy-aware mission handover approach,” *Under review / Preprint*, 2024.
- [2] H. Fouad and G. Beltrame, “Energy autonomy for robot systems with constrained resources,” *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3675–3690, 2022.
- [3] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions : Theory and applications,” *European Journal of Control*, vol. 49, pp. 346–357, 2019.
- [4] C. Pinciroli, V. Trianni, R. O’Grady, G. Pini, A. Brutschy, M. Soueidan, N. Mathews, E. Ferrante, G. A. Di Caro, F. Ducatelle, *et al.*, “ARGoS : a modular, multi-engine simulator for heterogeneous swarm robotics,” in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5027–5034, IEEE, 2011.
- [5] C. Pinciroli and G. Beltrame, “Buzz : An extensible programming language for heterogeneous swarm robotics,” in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3794–3800, IEEE, 2016.
- [6] ModalAI, Inc., “Seeker hardware quickstart and technical documentation.” <https://docs.modalai.com/seeker-hardware-quickstart/>, 2026. [En ligne ; consulté le 2 septembre 2026].
- [7] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, “Vehicle routing problems for drone delivery,” *IEEE Transactions on Systems, Man, and Cybernetics : Systems*, vol. 47, no. 1, pp. 70–85, 2017.
- [8] H. Fouad, V. S. Varadharajan, and G. Beltrame, “Energy sufficiency in unknown environments via control barrier functions,” *arXiv preprint arXiv :2306.15115*, 2023.
- [9] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA : MIT Press, 2005.
- [10] G. Raja, T. Mökkönen, and R. Ghabcheloo, “Safe robot control using occupancy grid map-based control barrier function (OGM-CBF),” *arXiv preprint arXiv :2405.10703*, 2024.
- [11] X. Sun, W. Yeoh, and S. Koenig, “Efficient incremental search for moving target search,” in *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 617–624, 2009.
- [12] M. Steinbrink, P. Koch, B. Jung, and S. May, “Rapidly-exploring random graph next-best view exploration for ground vehicles,” in *2021 European Conference on Mobile Robots (ECMR)*, pp. 1–7, IEEE, 2021.
- [13] T. Dang, F. Mascarich, S. Khattak, C. Papachristos, and K. Alexis, “Graph-based path planning for autonomous robotic exploration in subterranean environments,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3105–3112, IEEE, 2019.

- [14] N. Atanasov, J. Le Ny, K. Daniilidis, and G. J. Pappas, “Decentralized active information acquisition : Theory and application to multi-robot SLAM,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4775–4782, IEEE, 2015.
- [15] R. A. MacDonald and S. L. Smith, “Active sensing for motion planning in uncertain environments via mutual information policies,” *Autonomous Robots*, vol. 44, no. 7, pp. 1217–1233, 2020.

Annexe A : Auto-évaluation des compétences

Compétence 1 : Concevoir ou réaliser des solutions d'ingénierie, permettant de répondre à un cahier des charges

Élève-ingénieur : Auto-évaluation Niveau 1

► Contexte :

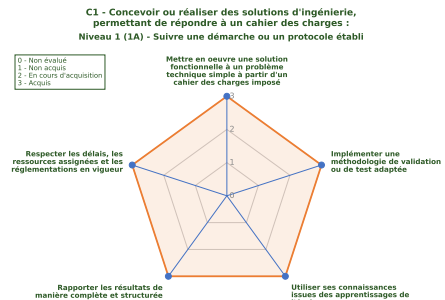
Durant mon stage au Laboratoire de vision et systèmes numériques (LVSN) de l'Université Laval, mes travaux visaient à garantir la sécurité d'une flotte de drones en intégrant des obstacles statiques au sein de fonctions barrières (*Control Barrier Functions*, CBF). Sans cahier des charges rigide imposé, l'objectif d'évitement et de sécurité était clairement défini.

► Argumentaire :

J'ai conçu et mis en œuvre la solution technique en autonomie (prise en compte géométrique du rayon des obstacles et adaptation de la distance de sécurité). J'ai systématiquement développé des tests automatisés avec Pytest et validé le comportement en simulation. Lors des réunions hebdomadaires, je restituais les résultats sous des formats adaptés (démonstrations en direct, présentations, synthèses orales).

► Conclusion :

Les délais hebdomadaires ont été rigoureusement tenus grâce aux prédictions temporelles précises de mon tuteur. Solution fonctionnelle validée avec succès. Niveau 1 pleinement acquis (3 partout).



Élève-ingénieur : Auto-évaluation Niveau 2

► Contexte :

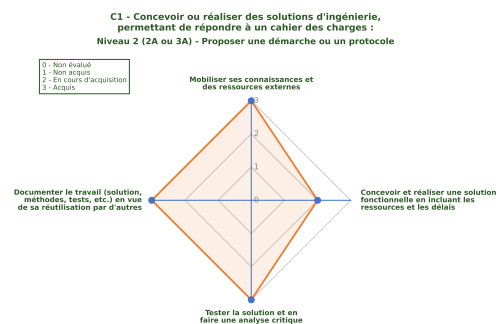
Dans le même cadre, en collaboration avec mon maître de stage et Hassan Fouad, nous avons cherché à dépasser les limitations d'une planification purement globale devenue trop lente pour la persistance de mission.

► Argumentaire :

En nous inspirant de travaux existants conçus pour l'exploration de robots terrestres, nous avons élaboré une démarche et un protocole adaptés à des drones aériens (architecture hybride globale/locale par coordination d'essaim et CBF). J'ai défini les démarches de vérification, conduit des campagnes de tests comparatifs et documenté le travail (rapport de stage, présentations techniques détaillées).

► Conclusion :

Autonomie confirmée sur la conception, l'expérimentation et la documentation (3 partout). En revanche, la tenue des délais est notée à 2 (en cours) : la tendance à sous-estimer le temps nécessaire aux intégrations complexes a repoussé la concrétisation sur plateforme physique en fin de stage.



Maître de stage : Remarques et observations (optionnel)

– Commentaires et observations éventuels de l'encadrant professionnel...

- **Règle de validation :** Le niveau est validé si tous les axes ont une notation = 3.
- Dans le cas contraire, le niveau n'est pas validé.

Annexe A : Auto-évaluation des compétences

Compétence 2 : Mettre en œuvre une démarche de recherche fondamentale ou appliquée à des fins d'innovation

Élève-ingénieur : Auto-évaluation Niveau 1

► Contexte :

Au laboratoire LVSN, le planificateur global initial présentait un temps de calcul prohibitif, empêchant la génération d'un chemin en un seul pas de simulation (*step*) et bloquant la réactivité du système multi-robots face aux dynamiques de l'environnement.

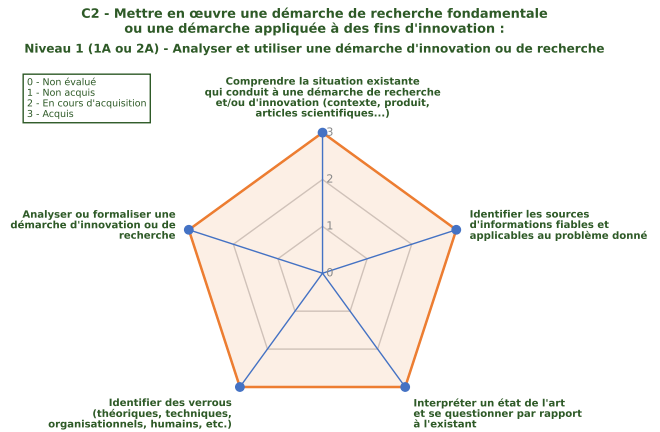
► Argumentaire :

Face à ce verrou technique, j'ai engagé une recherche documentaire ciblée dans les publications scientifiques récentes (algorithmes de graphes, Fringe Retrieving A*, EGO-Planner, champs de potentiel artificiels). J'ai analysé ces contributions théoriques et identifié une démarche éprouvée répondant précisément à nos contraintes d'exécution temps réel.

J'ai ensuite formalisé son adaptation au contexte de robots aériens évoluant sous observation partielle, puis implémenté et validé la solution en simulation, aboutissant à une réduction drastique du coût de calcul par rapport aux méthodes naïves.

► Conclusion :

Capacité démontrée à appréhender une problématique de recherche, exploiter une veille bibliographique et intégrer une solution innovante, rigoureuse et pérenne. Niveau 1 pleinement acquis (notation 3 sur tous les axes du référentiel).



Maître de stage : Remarques et observations (optionnel)

– Commentaires et observations éventuels de l'encadrant professionnel...

- **Règle de validation :** Le niveau est validé si tous les axes ont une notation = 3.
- Dans le cas contraire, le niveau n'est pas validé.

Élève-ingénieur : Auto-évaluation Niveau 1

► Contexte :

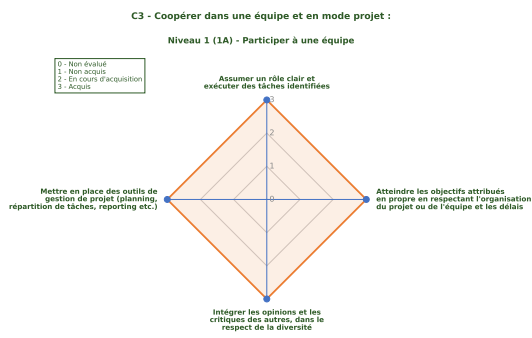
Projet de groupe 1A (Phelma / TWM) : conception et fabrication intégrale en 3 mois d'un méthaniseur avec son manuel d'utilisation au sein d'une équipe pluridisciplinaire de 7 étudiants (filières PET et PMP).

► Argumentaire :

Répartition par double rôle : managérial (responsable matériel : gestion, stockage de la caisse à outils durant 3 mois, approvisionnement à Saint-Égrève) et technique (responsable du circuit de gaz et comptage de bulles par vision par ordinateur adapté à mon profil PET, en synergie avec les élèves PMP sur la cuve et les bilans thermiques). Suivi rigoureux par diagramme de Gantt (GanttProject), Drive partagé et messagerie. Climat d'écoute active, aucun conflit et décisions consensuelles rapides.

► Conclusion :

Objectifs atteints dans le délai imparti de 3 mois, méthaniseur fonctionnel livré avec son manuel et maintenance réussie. Niveau 1 pleinement validé (notation 3 sur tous les axes).



Élève-ingénieur : Auto-évaluation Niveau 2

► Contexte :

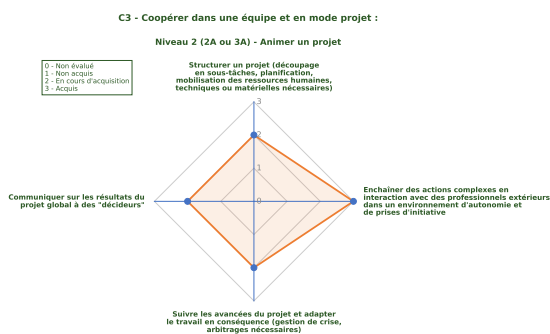
Au laboratoire LVSN, le projet de navigation coopérative a impliqué des interactions continues avec mon maître de stage (Hanqing Zhao) et Hassan Fouad.

► Argumentaire :

Participation active à la conduite du projet : découpage des objectifs en sous-tâches, propositions d'arbitrages techniques face aux blocages algorithmiques et animation des points de restitution hebdomadaires (démonstrations, présentations et synthèses orales pour les décideurs).

► Conclusion :

L'autonomie relationnelle et la communication technique auprès des décideurs sont acquises. La planification globale et l'estimation des délais restent en cours d'acquisition (notées 2), avec une marge de progression identifiée pour anticiper les imprévus techniques en 3A.



Maître de stage : Remarques et observations (optionnel)

– Commentaires et observations éventuels de l'encadrant professionnel...

- **Règle de validation :** Le niveau est validé si tous les axes ont une notation = 3.
- Dans le cas contraire, le niveau n'est pas validé.

Élève-ingénieur : Auto-évaluation Niveau 1**► Contexte :**

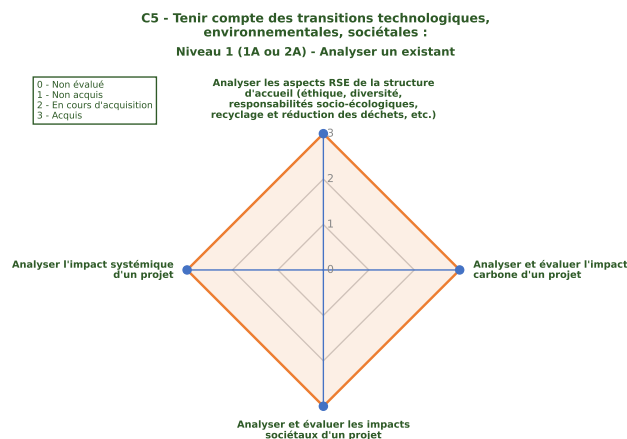
Analyse DDRSE de la structure d'accueil (laboratoire LVSN, Université Laval à Québec) et évaluation de l'impact global de mes activités de stage.

► Argumentaire :

J'ai pu observer une démarche RSE exemplaire au sein du campus de l'Université Laval : tri sélectif strict à trois voies (déchets ultimes en poubelle noire, bac de recyclage, bac de compostage), campagnes régulières de sensibilisation aux économies d'énergie (extinction des lumières et des postes de travail), et promotion active des mobilités durables (partenariat avec les vélos électriques en libre-service à Vélo et tarification incitative avec le réseau de transport en commun de Québec, le RTC). Sur le plan carbone, l'activité de recherche numérique en elle-même est très sobre : réunions en personne ou visios légères, simulations locales sur PC de laboratoire sans cluster énergivore, alimenté par l'hydroélectricité québécoise décarbonée. L'impact carbone prépondérant réside dans le vol long-courrier aller-retour France-Québec (environ 1,9 tCO₂e, détaillé en Annexe B). Sur le plan humain et sociétal, l'accueil au laboratoire, la mixité culturelle et l'esprit d'entraide ont été remarquables.

► Conclusion :

Prise de conscience systémique des enjeux environnementaux et sociétaux de la recherche. Niveau 1 pleinement acquis (notation 3 sur l'ensemble des axes).

**Maître de stage : Remarques et observations (optionnel)**

– Commentaires et observations éventuels de l'encadrant professionnel...

- **Règle de validation :** Le niveau est validé si tous les axes ont une notation = 3.
- Dans le cas contraire, le niveau n'est pas validé.

Élève-ingénieur : Auto-évaluation Niveau 2

► Contexte :

Conception logicielle et choix d'architecture pour le simulateur multi-robots au laboratoire LVSN dans une optique de frugalité numérique et de durabilité.

► Argumentaire :

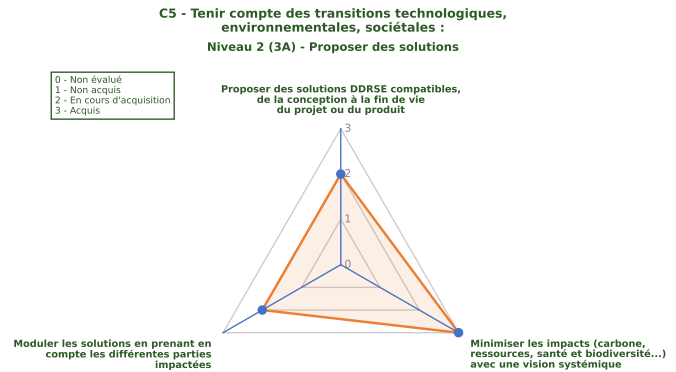
Pour limiter la consommation énergétique des calculs sans dégrader la réactivité du système, j'ai adopté des principes d'écoconception logicielle :

- Activation conditionnelle du solveur QP/CBF uniquement au voisinage critique des obstacles pour éviter des optimisations inutiles à chaque pas de temps ;
- Recours systématique aux opérations vectorisées sous NumPy et précalcul d'un champ de distance signé (SDF) évitant des requêtes géométriques répétées ;
- Réflexion sur l'impact de cycle de vie et perspective de portage en C++ ou Rust pour optimiser l'efficacité énergétique lors du déploiement final sur microcontrôleurs embarqués.

Sur le plan sociétal, l'application vise à sécuriser le déploiement de drones autonomes pour des missions de surveillance environnementale ou de secours en zone sinistrée.

► Conclusion :

Intégration concrète de la sobriété algorithmique dans la conception d'un système robotique. Compétence en cours de déploiement (notée 2 et 3), constituant un socle solide pour les projets d'ingénierie durable de 3A.



Maître de stage : Remarques et observations (optionnel)

– Commentaires et observations éventuels de l'encadrant professionnel...

- **Règle de validation :** Le niveau est validé si tous les axes ont une notation = 3.
- Dans le cas contraire, le niveau n'est pas validé.

B Considération des impacts écologiques et sociétaux

B.1 Analyse de l'impact environnemental et sociétal

B.1.1 Calcul de l'empreinte carbone

L'évaluation de l'empreinte carbone générée au cours des 16 semaines de stage au laboratoire LVSN (Université Laval) a été menée en distinguant les activités directes de recherche au laboratoire et les flux liés à la mobilité et à la vie courante. Les données de référence s'appuient sur la *Base Empreinte* de l'ADEME et les métriques fournies par *Hydro-Québec*.

Le tableau 3 synthétise les postes d'émissions identifiés. On observe que l'impact énergétique direct des activités informatiques de simulation est remarquablement faible (inférieur à 0,2 kgCO₂e), principalement grâce à l'approvisionnement du réseau québécois par une hydroélectricité décarbonée dont le facteur d'émission est estimé à environ 1,3 gCO₂e/kWh (contre une moyenne européenne de plus de 250 gCO₂e/kWh).

Le poste prépondérant et écrasant demeure le voyage aérien international aller-retour entre la France et le Québec (environ 11 500 km cumulés), représentant à lui seul plus de 83 % des émissions totales sur la période.

TABLE 3 – Synthèse de l'empreinte carbone estimée sur la période de stage (16 semaines).

Poste d'émission	Émissions (kgCO ₂ e)	Hypothèses et données sources
Matériel informatique	0,1	Station de travail labo (100 W), 7 h/j sur 80 jours ($\approx$ 56 kWh), mix Hydro-Québec (1,3 gCO ₂ e/kWh).
Serveurs et calculs	0,1	Simulations locales Pygame/NumPy sans cluster externe ($\approx$ 25 kWh).
Transports domicile-travail	0	Déplacements quotidiens en vélo
Trajets longue distance	1 900	Vol aller-retour France – Québec (classe éco, simulateur ADEME / DGAC avec forçage radiatif).
Vie sur place	350	Hébergement, alimentation locale, chauffage et consommation domestique.
Total estimé	2 250	Bilan global sur les 16 semaines de stage ($\approx$ 2,3 tCO₂e)

B.1.2 Initiatives personnelles et professionnelles d'atténuation

Afin de contenir au maximum cette empreinte écologique au quotidien, plusieurs démarches écoresponsables ont été appliquées :

- **Écomobilité urbaine** : Renoncement total à l'automobile individuelle sur place, au profit de l'usage d'un vélo, prêté par mon propriétaire.

- **Sobriété des postes informatiques** : Extinction complète et systématique de la station de travail, des écrans et des bancs de test lors des pauses prolongées, en soirée et le week-end.
- **Gestion rigoureuse des déchets et alimentation** : Utilisation stricte des filières de tri et de compostage mises en place sur le campus, et consommation privilégiée de produits frais locaux en circuit court.
- **Frugalité expérimentale** : Limitation du nombre d'exécutions de sweep de paramètres intensifs grâce à une validation préalable par des tests unitaires ciblés (`pytest`) et des jeux d'essais réduits.

B.1.3 Analyse de la structure d'accueil (politique DDRSE)

L'Université Laval et le Département de génie électrique et de génie informatique se distinguent par une maturité environnementale remarquable, l'université étant certifiée au niveau *STARS* (*Sustainability Tracking, Assessment & Rating System*) pour ses engagements durables :

- **Politique des 3 voies de tri** : Chaque étage du Pavillon Adrien-Pouliot dispose d'îlots de tri comprenant une poubelle noire (déchets ultimes destinés à l'enfouissement), un bac bleu/vert de recyclage (matières sèches) et un bac brun de compostage pour l'intégralité des résidus organiques.
- **Efficacité énergétique et sobriété des bâtiments** : Campagnes visuelles récurrentes et dispositifs de sensibilisation invitant les usagers à éteindre les éclairages et les appareils numériques inutilisés.
- **Incitations aux mobilités durables** : Partenariat stratégique avec la Ville de Québec pour le déploiement de stations de vélos électriques *à Vélo* directement sur le campus et tarifs incitatifs pour les cartes de transport RTC.
- **Éthique et climat social** : Le laboratoire LVSN offre un environnement multiculturel bienveillant, favorisant la mixité, l'accueil d'étudiants internationaux et l'esprit d'entraide interdisciplinaire.

B.1.4 Analyse de l'utilité sociétale et scientifique du projet

Le développement d'algorithmes garantissant mathématiquement l'absence de collision via des fonctions barrières (*Control Barrier Functions*) répond à un enjeu sociétal de premier plan :

- **Sécurisation des opérations autonomes** : La possibilité de garantir formellement l'évitement d'obstacles statiques et dynamiques est un prérequis indispensable à l'insertion sécurisée de flottes de drones dans l'espace aérien civil.
- **Missions d'intérêt général** : Les retombées directes de ces travaux concernent la recherche et le sauvetage en zone sinistrée (secours post-catastrophes naturelles), l'inspection d'infrastructures critiques (ponts, barrages, lignes électriques haute tension) et la surveillance pérenne de feux de forêt ou d'écosystèmes protégés, en évitant d'exposer des vies humaines à des tâches périlleuses.

B.1.5 Impacts positifs et négatifs (vision systémique)

Une analyse systémique met en regard les gains et les coûts environnementaux de cette technologie :

- **Impacts positifs** : Substitution de véhicules terrestres ou d'hélicoptères lourds très émetteurs par des micro-drones électriques légers ; détection précoce d'anomalies environnementales évitant des sinistres de grande ampleur ; minimisation des pertes matérielles dues aux collisions.
- **Impacts négatifs et effets rebonds** : Fabrication des vecteurs robotiques mobilisant des composants électroniques complexes, des métaux rares et des batteries au lithium dont le recyclage reste imparfait ; risque d'effet rebond consistant à multiplier massivement les vols de drones sous prétexte d'un coût marginal faible, entraînant des nuisances sonores et une sollicitation accrue des ressources minérales.

Résumé

La surveillance autonome et pérenne de vastes territoires par des essaims de drones requiert de concilier couverture continue, autonomie énergétique limitée et garanties formelles de sécurité sous observabilité partielle. Réalisé au Laboratoire de Vision et Systèmes Numériques (LVSN) de l'Université Laval, ce projet d'ingénierie s'est articulé autour de deux axes complémentaires.

Dans un premier volet entièrement abouti, nous avons conçu et validé un simulateur multi-robots synchrone, modulaire et déterministe. L'architecture logicielle développée intègre une couche d'abstraction matérielle (HAL), une modélisation fine de la cinématique et de la décharge/recharge de batterie, une cartographie locale par grille d'occupation (OGM), le calcul en temps réel de champs de distance signée (SDF) et un ordonnancement strict par barrière IPC (*lockstep*). Cette infrastructure a fait l'objet d'une validation métrologique et logicielle rigoureuse.

Dans un second volet méthodologique, nous avons formalisé une architecture de coordination décentralisée (langage Buzz, table d'état distribuée), un planificateur local spatio-temporel sur graphe RRG par échantillonnage biaisé et scoring multi-objectifs, ainsi qu'un filtre cinématique réactif d'évitement d'obstacles par *Control Barrier Functions* (CBF). Les campagnes expérimentales conduites sur ce prototype ont permis de caractériser les dynamiques de mission et de diagnostiquer les limites d'un réglage heuristique des poids (effet de falaise énergétique et conflits de proximité inter-drones). Ces résultats établissent une feuille de route technique claire pour le couplage en boucle fermée du repli incrémental FRA* et des barrières réciproques multi-robots.

Mots-clés : Systèmes multi-robots, Simulation décentralisée, Observabilité partielle, *Control Barrier Functions* (CBF), Évitement d'obstacles, Planification sur graphe, Coordination décentralisée, Persistance de mission.

Abstract

Persistent autonomous monitoring of large-scale environments using UAV swarms requires reconciling continuous coverage, limited battery life, and formal safety guarantees under partial observability. Conducted at the Computer Vision and Systems Laboratory (LVSN) at Université Laval, this engineering project focuses on two complementary axes.

First, we designed, fully implemented, and validated a synchronous, modular, and deterministic multi-robot simulator. The software architecture incorporates a hardware abstraction layer (HAL), kinematic and battery dynamics modeling, local occupancy grid mapping (OGM), real-time signed distance field (SDF) generation, and a strict IPC lockstep barrier scheduler. This framework underwent thorough quality assurance and validation.

Second, we formulated a decentralized coordination architecture (Buzz language, distributed state sharing), a spatio-temporal local RRG graph planner with biased sampling and multi-objective scoring, and a reactive obstacle avoidance filter based on Control Barrier Functions (CBF). Sweep campaigns on this intermediate prototype revealed key flight dynamics and diagnosed the failure modes of heuristic tuning (battery cliff effect, inter-drone proximity violations). These findings establish a solid technical roadmap for closed-loop incremental FRA* energy coupling and reciprocal multi-robot barrier constraints.

Keywords : Multi-robot systems, Decentralized simulation, Partial observability, Control Barrier Functions (CBF), Obstacle avoidance, Graph planning, Decentralized coordination, Mission persistence.